Lab 11

Open Ended Lab

https://github.com/mmujtaba25/cs-212

Muhammad Mujtaba

CMD ID: 540040

mmujtaba.bese25seecs@seecs.edu.pk

Class: BESE 16B

Batch: 2k25

Lab 11 - App - Base - Rateable

```java
package Lab11.App.Base;

public interface Rateable
{
    void rate(double newRating);

    Rating getRating();
}
```

Lab 11 - App - Base - Rating

```java
package Lab11.App.Base;

import java.util.Date;
import java.util.HashMap;
import java.util.Map;

public class Rating
{
    public static final double MIN_RATING = 0;
    public static final double MAX_RATING = 10;

    private final double initialRating;
    private double currentRating;

    private final Map<Date, Double> ratingsHistory = new HashMap<>();
    private int ratingCount = 0;

    public Rating(double initialRating)
    {
        this.initialRating = initialRating;
        rate(initialRating);
    }

    public double latest() { return currentRating; }

    public double average()
    {
        return
ratingsHistory.values().stream().mapToDouble(Double::doubleValue).average().orElse(0);
    }

    public double minimum()
    {
        return
ratingsHistory.values().stream().mapToDouble(Double::doubleValue).min().orElse(0);
    }

    public Rating rate(double newRating)
    {
        this.currentRating = Math.clamp(newRating, MIN_RATING, MAX_RATING);
        ratingCount++;
        ratingsHistory.put(new Date(), newRating);
        return this;
    }

    public Rating increment(double amount) { return rate(initialRating + amount); }

    public int getRatingCount() { return ratingCount; }

    public Map<Date, Double> getRatingsHistory() { return ratingsHistory; }

    /* UTILITIES */
```

```
    public String getFormatted()
    {
        return switch ((int) (Math.floor(currentRating) / 2))
        {
            //@formatter:off
            case 0  -> "[ - - - - - ]";
            case 1  -> "[ * - - - - ]";
            case 2  -> "[ * * - - - ]";
            case 3  -> "[ * * * - - ]";
            case 4  -> "[ * * * * - ]";
            case 5  -> "[ * * * * * ]";
            default -> "[  INVALID  ]";
            //@formatter:on
        };
    }
}
```

Lab 11 - App - Base - Chef

```java
package Lab11.App.Base;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public abstract class Chef implements Rateable
{
    private static long _lastID = 0;
    private final long id = _lastID++;

    private final String name;
    private final List<Recipe> recipes = new ArrayList<>();

    private final Rating rating = new Rating(0);

    public abstract int maxRecipes();

    public Chef(String name)
    {
        this.name = name;
    }

    /* RECIPE */

    public boolean addRecipe(Recipe recipe)
    {
        if (recipes.size() ≥ maxRecipes()) return false;

        recipes.add(recipe);
        return true;
    }

    /* RATEABLE */

    @Override
    public void rate(double newRating) { rating.rate(newRating); }

    @Override
    public Rating getRating() { return rating; }

    /* GETTERS & SETTERS */

    public long getId() { return id; }

    public String getName() { return name; }

    public List<Recipe> getRecipes() { return recipes; }

    /* OBJECT OVERRIDE */

    @Override
```

```
    public boolean equals(Object obj)
    {
        // same if id is same
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;

        Chef chef = (Chef) obj;
        return id == chef.id;
    }

    @Override
    public int hashCode()
    {
        return Objects.hash(id);
    }

    @Override
    public String toString()
    {
        return String.format(
                "%s[id=%d, name='%s', recipes=%d, rating=%.1f]",
                this.getClass().getSimpleName(),
                id,
                name,
                recipes.size(),
                rating.latest()
        );
    }
}
```

Lab 11 - App - JuniorChef

```java
package Lab11.App;

import Lab11.App.Base.Chef;

public class JuniorChef extends Chef
{
    private final SeniorChef supervisor;

    public JuniorChef(String name, SeniorChef supervisor)
    {
        super(name);
        this.supervisor = supervisor;
    }

    @Override
    public int maxRecipes() { return 1; }

    /* GETTERS */

    public SeniorChef getSupervisor() { return supervisor; }
}
```

Lab 11 - App - SeniorChef

```java
package Lab11.App;

import Lab11.App.Base.Chef;

public class SeniorChef extends Chef
{
    private final int experienceInYears;

    public SeniorChef(String name, int experienceInYears)
    {
        super(name);
        this.experienceInYears = experienceInYears;
    }

    @Override
    public int maxRecipes() { return 3; }

    /* RATING */

    @Override
    public void rate(double newRating)
    {
        // rate based on experience
        double adjustedRating = newRating * (1 + experienceInYears / 10.0); // more
experience = higher rating
        super.rate(adjustedRating);
    }

    /* GETTERS */

    public int getExperienceInYears() { return experienceInYears; }
}
```

Lab 11 - App - Base - Recipe

```java
package Lab11.App.Base;

import java.util.List;
import java.util.Objects;

public class Recipe implements Rateable
{
    private final String name;
    private final String description;
    private final List<String> ingredients;
    private final String instructions;

    private final Rating rating = new Rating(0);

    public Recipe(String name, String description, List<String> ingredients, String
instructions)
    {
        this.name = name;
        this.description = description;
        this.ingredients = ingredients;
        this.instructions = instructions;
    }

    @Override
    public void rate(double newRating) { rating.rate(newRating); }

    /* GETTERS & SETTERS */

    public String getName() { return name; }

    public String getDescription() { return description; }

    public List<String> getIngredients() { return ingredients; }

    public String getInstructions() { return instructions; }

    @Override
    public Rating getRating() { return rating; }

    /* OBJECT OVERRIDE */

    @Override
    public boolean equals(Object obj)
    {
        // same if all fields are same
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;

        Recipe recipe = (Recipe) obj;
        return name.equals(recipe.name) &&                  //
                description.equals(recipe.description) &&   //
                ingredients.equals(recipe.ingredients) &&   //
```

```
                instructions.equals(recipe.instructions);
    }

    @Override
    public int hashCode()
    {
        return Objects.hash(name, description, ingredients, instructions);
    }

    @Override
    public String toString()
    {
        return String.format("Recipe[name='%s', rating=%.1f]", name,
rating.latest());
    }
}
```

Lab 11 - App - CookingContest

```java
package Lab11.App;

import Lab11.App.Base.Chef;
import Lab11.App.Base.Recipe;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class CookingContest
{
    private final List<Chef> participants = new ArrayList<>();

    /* CHEFS */

    public void addSeniorChef(String name, int experienceInYears)
    {
        participants.add(new SeniorChef(name, experienceInYears));
    }

    public void addJuniorChef(String name, SeniorChef supervisor)
    {
        participants.add(new JuniorChef(name, supervisor));
    }

    public Optional<Chef> getChef(String name)
    {
        return participants.stream().filter(chef ->
chef.getName().equals(name)).findFirst();
    }
}
```

Lab 11 - App - Main

```java
package Lab11.App;

import Lab11.App.Base.Chef;
import Lab11.App.Base.Recipe;

import java.util.List;

public class Main
{
    public static void main(String[] args)
    {
        CookingContest contest = new CookingContest();

        // Add chefs using simple prototype names.
        contest.addSeniorChef("SC-1", 12);
        contest.addSeniorChef("SC-2", 7);

        SeniorChef seniorChef1 = (SeniorChef) contest.getChef("SC-1").orElseThrow();
        SeniorChef seniorChef2 = (SeniorChef) contest.getChef("SC-2").orElseThrow();

        contest.addJuniorChef("JC-1", seniorChef1);
        contest.addJuniorChef("JC-2", seniorChef2);

        JuniorChef juniorChef1 = (JuniorChef) contest.getChef("JC-1").orElseThrow();
        JuniorChef juniorChef2 = (JuniorChef) contest.getChef("JC-2").orElseThrow();

        // Add recipes and show the max-recipes limit in action.
        Recipe recipe1 = new Recipe(
                "Recipe 1",
                "Demo recipe for Senior Chef 1.",
                List.of("ingredient 1", "ingredient 2", "ingredient 3"),
                "1. Prep ingredients. 2. Cook everything. 3. Plate the dish."
        );
        Recipe recipe2 = new Recipe(
                "Recipe 2",
                "Second demo recipe for Senior Chef 1.",
                List.of("ingredient 4", "ingredient 5"),
                "1. Mix ingredients. 2. Finish and serve."
        );
        Recipe recipe3 = new Recipe(
                "Recipe 3",
                "Third demo recipe for Senior Chef 1.",
                List.of("ingredient 6", "ingredient 7", "ingredient 8"),
                "1. Combine. 2. Heat. 3. Serve."
        );
        Recipe recipe4 = new Recipe(
                "Recipe 4",
                "This one should exceed Senior Chef 1's recipe limit.",
                List.of("ingredient 9", "ingredient 10"),
                "1. Try to add it. 2. Observe the failure."
        );
```

```
        Recipe recipe5 = new Recipe(
                "Recipe 5",
                "First demo recipe for Senior Chef 2.",
                List.of("ingredient 11", "ingredient 12"),
                "1. Chop. 2. Cook. 3. Taste."
        );
        Recipe recipe6 = new Recipe(
                "Recipe 6",
                "Second demo recipe for Senior Chef 2.",
                List.of("ingredient 13", "ingredient 14", "ingredient 15"),
                "1. Prepare. 2. Bake. 3. Rest."
        );
        Recipe recipe7 = new Recipe(
                "Recipe 7",
                "Third demo recipe for Senior Chef 2.",
                List.of("ingredient 16", "ingredient 17"),
                "1. Simmer. 2. Finish."
        );

        Recipe juniorRecipe1 = new Recipe(
                "Recipe 8",
                "Junior Chef 1's only recipe.",
                List.of("ingredient 18", "ingredient 19"),
                "1. Simple prep. 2. Simple cook."
        );
        Recipe juniorRecipe2 = new Recipe(
                "Recipe 9",
                "Junior Chef 2's only recipe.",
                List.of("ingredient 20", "ingredient 21"),
                "1. Quick prep. 2. Quick cook."
        );

        printSection("ADDING RECIPES");
        printAddResult("SC-1", seniorChef1.addRecipe(recipe1));
        printAddResult("SC-1", seniorChef1.addRecipe(recipe2));
        printAddResult("SC-1", seniorChef1.addRecipe(recipe3));
        printAddResult("SC-1", seniorChef1.addRecipe(recipe4));

        printAddResult("SC-2", seniorChef2.addRecipe(recipe5));
        printAddResult("SC-2", seniorChef2.addRecipe(recipe6));
        printAddResult("SC-2", seniorChef2.addRecipe(recipe7));

        printAddResult("JC-1", juniorChef1.addRecipe(juniorRecipe1));
        printAddResult(
                "JC-1", juniorChef1.addRecipe(new Recipe(
                        "Recipe 10",
                        "This one should exceed Junior Chef 1's recipe limit.",
                        List.of("ingredient 22"),
                        "1. Confirm the limit."
                ))
        );

        printAddResult("JC-2", juniorChef2.addRecipe(juniorRecipe2));
        printAddResult(
```

```
                "JC-2", juniorChef2.addRecipe(new Recipe(
                        "Recipe 11",
                        "This one should exceed Junior Chef 2's recipe limit.",
                        List.of("ingredient 23"),
                        "1. Confirm the limit."
                ))
        );

        // Rate chefs and recipes.
        printSection("RATING CHEFS & RECIPES");
        seniorChef1.rate(8.0);
        seniorChef1.rate(9.0);
        seniorChef1.getRating().increment(1.0);
        seniorChef2.rate(7.5);
        seniorChef2.rate(8.0);

        juniorChef1.rate(6.0);
        juniorChef1.getRating().increment(0.5);
        juniorChef2.rate(6.5);

        recipe1.rate(9.0);
        recipe1.rate(9.5);
        recipe2.rate(8.0);
        recipe3.getRating().increment(7.5);
        recipe5.rate(7.0);
        recipe6.rate(8.5);
        juniorRecipe1.rate(6.5);
        juniorRecipe2.rate(7.2);

        // Print a simple prototype-style walkthrough of the contest state.
        printSection("COOKING CONTEST DEMO");
        printChefDetails("SC-1", seniorChef1);
        printChefDetails("SC-2", seniorChef2);
        printChefDetails("JC-1", juniorChef1);
        printChefDetails("JC-2", juniorChef2);

        printSection("RATINGS SUMMARY");
        printRatingDetails("SC-1 : rating", seniorChef1.getRating());
        printRatingDetails("SC-2 : rating", seniorChef2.getRating());
        printRatingDetails("JC-1 : rating", juniorChef1.getRating());
        printRatingDetails("JC-2 : rating", juniorChef2.getRating());
        printRatingDetails("RECIPE 1 : rating", recipe1.getRating());
        printRatingDetails("RECIPE 3 : rating", recipe3.getRating());
        printRatingDetails("RECIPE 8 : rating", juniorRecipe1.getRating());
    }

    private static void printSection(String title)
    {
        System.out.println("\n" + "=".repeat(70));
        System.out.println("  " + title);
        System.out.println("=".repeat(70));
    }

    private static void printAddResult(String chefName, boolean added)
```

```
    {
        String status = added ? " Added" : " Rejected";
        System.out.printf("  %-20s → %s%n", chefName, status);
    }

    private static void printChefDetails(String label, Chef chef)
    {
        System.out.println("\n  ┌─ " + label);
        System.out.println("  │  Name:      " + chef.getName());
        System.out.println("  │  ID:        " + chef.getId());
        System.out.println("  │  Recipes:   " + chef.getRecipes().size() + " / " +
chef.maxRecipes());

        if (chef instanceof SeniorChef seniorChef)
        {
            System.out.println("  │  Experience: " +
seniorChef.getExperienceInYears() + " years");
        }
        else if (chef instanceof JuniorChef juniorChef)
        {
            System.out.println("  │  Supervisor: " +
juniorChef.getSupervisor().getName());
        }

        if (!chef.getRecipes().isEmpty())
        {
            System.out.println("  │");
            List<Recipe> recipes = chef.getRecipes();
            for (int i = 0; i < recipes.size(); i++)
            {
                boolean isLast = i == recipes.size() - 1;
                String prefix = isLast ? "  └─" : "  ├─";
                Recipe recipe = recipes.get(i);
                System.out.println(prefix + " " + recipe.getName() + " | Rating: " +
recipe.getRating().latest());
            }
        }
        else
        {
            System.out.println("  └─ (no recipes)");
        }
    }

    private static void printRatingDetails(String label, Lab11.App.Base.Rating
rating)
    {
        System.out.println("\n> " + label + " : " + rating.getFormatted());
        System.out.printf("     Latest:     %.2f%n", rating.latest());
        System.out.printf("     Average:    %.2f%n", rating.average());
        System.out.printf("     Count:      %d%n", rating.getRatingCount());
    }
}
```

Program Output

```
═══════════════════════════════════════════════════════════════
  ADDING RECIPES
═══════════════════════════════════════════════════════════════
  SC-1            →  Added
  SC-1            →  Added
  SC-1            →  Added
  SC-1            →  Rejected
  SC-2            →  Added
  SC-2            →  Added
  SC-2            →  Added
  JC-1            →  Added
  JC-1            →  Rejected
  JC-2            →  Added
  JC-2            →  Rejected

═══════════════════════════════════════════════════════════════
  RATING CHEFS & RECIPES
═══════════════════════════════════════════════════════════════


═══════════════════════════════════════════════════════════════
  COOKING CONTEST DEMO
═══════════════════════════════════════════════════════════════

  ┌─ SC-1
  │  Name:  SC-1
  │  ID:    0
  │  Recipes:   3 / 3
  │  Experience: 12 years
  │
  ├─ Recipe 1 | Rating: 9.5
  ├─ Recipe 2 | Rating: 8.0
  └─ Recipe 3 | Rating: 7.5

  ┌─ SC-2
  │  Name:  SC-2
  │  ID:    1
  │  Recipes:   3 / 3
  │  Experience: 7 years
  │
  ├─ Recipe 5 | Rating: 7.0
  ├─ Recipe 6 | Rating: 8.5
  └─ Recipe 7 | Rating: 0.0

  ┌─ JC-1
  │  Name:  JC-1
  │  ID:    2
  │  Recipes:   1 / 1
  │  Supervisor: SC-1
  │
  └─ Recipe 8 | Rating: 6.5
```

```
  ┌─ JC-2
  │  Name:  JC-2
  │  ID:    3
  │  Recipes:   1 / 1
  │  Supervisor: SC-2
  │
  └─ Recipe 9 | Rating: 7.2

═══════════════════════════════════════════════════════════════
  RATINGS SUMMARY
═══════════════════════════════════════════════════════════════

> SC-1 : rating : [ - - - - - ]
     Latest:     1.00
     Average:    0.50
     Count:  4

> SC-2 : rating : [ * * * * * ]
     Latest:     10.00
     Average:    6.80
     Count:  3

> JC-1 : rating : [ - - - - - ]
     Latest:     0.50
     Average:    0.25
     Count:  3

> JC-2 : rating : [ * * * - - ]
     Latest:     6.50
     Average:    3.25
     Count:  2

> RECIPE 1 : rating : [ * * * * - ]
     Latest:     9.50
     Average:    4.75
     Count:  3

> RECIPE 3 : rating : [ * * * - - ]
     Latest:     7.50
     Average:    3.75
     Count:  2

> RECIPE 8 : rating : [ * * * - - ]
     Latest:     6.50
     Average:    3.25
     Count:  2
```